

## Understanding the code without quantum mathematics

The data section selects only Setosa and Versicolor, and keeps petal length and width. Standardisation prevents the two measurements from being treated on incompatible scales. The arctangent bounds the transformed values so they remain convenient rotation inputs.

The device named default.qubit is a simulator, so no physical quantum computer or cloud account is required. AngleEmbedding rotates the two qubits according to the two flower features. BasicEntanglerLayers supplies trainable rotations and an interaction between qubits. The two measurements returned by the QNode become two ordinary numbers.

TorchLayer packages the QNode as a PyTorch module. From this point onwards, PyTorch can place the quantum layer inside a sequential model, calculate binary cross-entropy loss and update all trainable parameters with Adam. The developer does not need to create a separate optimisation routine for the quantum part.

## Experimental results

With the fixed random seed and the stated 75/25 stratified split, the training loss fell from 0.7097 in the first epoch to 0.1058 after 50 epochs. Training accuracy reached 100 per cent by epoch 10. On the held-out test set, all 25 flowers were classified correctly: 13 Setosa and 12 Versicolor as shown in Figure 4.

## Understanding the results

This trained model correctly classified all 25 test flowers in this split as shown in Figure 5. This does not mean that PennyLane always produces perfect models, that every Iris split will produce the same result, or that quantum computing has outperformed classical machine learning. Setosa and Versicolor are easily separated using petal measurements, and classical models can also solve this small task very well.

The value of this experiment is educational: it exposes every stage of a hybrid QML workflow in a problem that runs quickly on a laptop.

A serious comparison would require repeated splits, multiple random seeds, classical baselines, noise experiments, runtime and resource measurements,

```
(.venv-iris) C:\Users\Admin-SSD\pennylane-iris>python iris_pennylane_qml.py
Experiment: Iris Setosa vs Versicolor
Training samples: 75 | Test samples: 25
Features: petal length and petal width | Qubits: 2
0: - (AngleEmbedding(M0)-BasicEntanglerLayers(M1)- <Z>
1: - (AngleEmbedding(M0)-BasicEntanglerLayers(M1)- <Z>

M0 =
tensor([0.7420, 0.9123])
M1 =
tensor([[0., 0.],
        [0., 0.]])
Epoch 01 | loss=0.7097 | train accuracy=0.493
Epoch 10 | loss=0.5473 | train accuracy=1.000
Epoch 20 | loss=0.3481 | train accuracy=1.000
Epoch 30 | loss=0.2166 | train accuracy=1.000
Epoch 40 | loss=0.1478 | train accuracy=1.000
Epoch 50 | loss=0.1058 | train accuracy=1.000

Final test accuracy: 1.000 (25/25)
Confusion matrix:
[[13  0]
 [ 0 12]]
Results saved in: C:\Users\Admin-SSD\pennylane-iris\iris_qml_results
```

Figure 4: Results of the experiment

| Experiment setting    | Observed value                                     |
|-----------------------|----------------------------------------------------|
| Training/test samples | 75 / 25                                            |
| Qubits                | 2                                                  |
| Epochs                | 50                                                 |
| Final training loss   | 0.1058                                             |
| Test accuracy         | 100% (25/25)                                       |
| Confusion matrix      | 13 Setosa correct; 12 Versicolor correct; 0 errors |

Table 2: Reproducible result obtained with the listed split, seed and training configuration

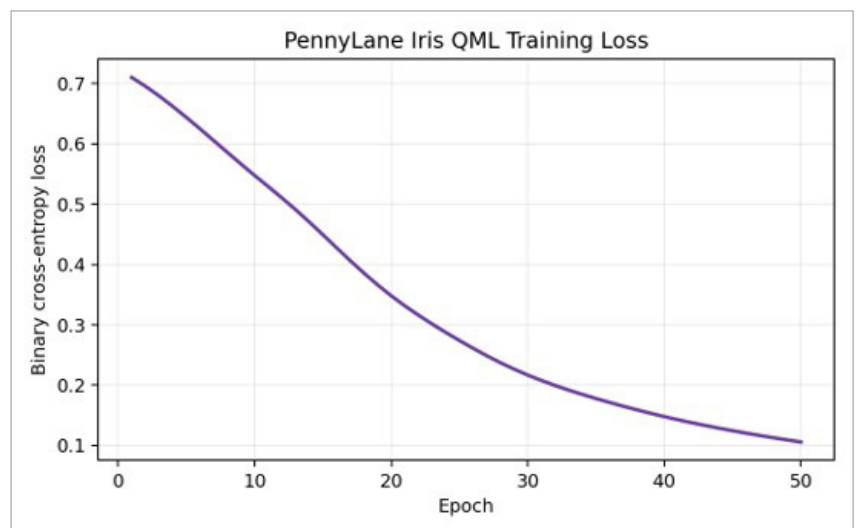


Figure 5(a): QML training loss across 50 epochs